Final relation — Sentence splitting system

This report summarizes the entire work carried out: data preparation, implementation of the two system versions, evaluation method, error analysis, and final comparison of results.

All project files are available at the following GitHub repository:
https://github.com/artibaniavvolgibili/MNLP/tree/main/sentence_splitting_project

1. Project objective

The objective was to build a sentence splitting system, that is, a system capable of correctly dividing a continuous text into sentences. The work was developed aiming to predict the most basic sentence boundaries, namely through punctuation analysis.

2. Dataset preparation

The original dataset files contained the special marker <EOS>, which indicated the end of a sentence. For evaluation, it was necessary to build a clean gold dataset with one sentence per line. For this purpose, a cleaning script was used that replaced each <EOS> with a newline, then removing empty lines and unnecessary spaces.

The project structure was organized as follows:

data/raw → original dataset with <EOS>

data/processed → clean gold, one sentence per line

output/sentence_split_versione1 → output of Version 1

output/sentence_split_versione2 → output of Version 2

3. Version 1: classical punctuation baseline

Version 1 was written in a simple style. The basic idea is very straightforward: the text is read from file, the <EOS> marker is removed, and the content is segmented after punctuation marks . ! ? followed by a space.

Segmentation rule used in Version 1:

```
re.split(r'(?<=[.!?])\s+', testo)
```

This version was already able to achieve good results on both datasets (Italian and English). A total of 6 text files.

4. Evaluation of Version 1

The evaluation was performed by comparing the system output with the gold in data/processed. For each file, three widely used metrics were computed: precision, recall, and F1. Precision measures the proportion of predicted sentences that are correct, while Recall measures the proportion of correct sentences that the system is able to identify. The F1 score combines both metrics to provide a balanced evaluation of the model's performance.

Precision = veri_positivi / tot_pred

Recall = frasi corrette trovate / frasi totali nel gold

F1 = 2 * (Precision * Recall) / (Precision + Recall)

Risult of Versione 1

File	Precision	Recall	F1	Note
en_partut-ud-dev.sent_split	0.9601	0.9399	0.9499	Good baseline
en_partut-ud-test.sent_split	0.9947	0.9894	0.9920	High
en_partut-ud-train.sent_split	0.9631	0.9425	0.9527	Stable
it_partut-ud-dev.sent_split	0.9527	0.9253	0.9388	Not optimal
it_partut-ud-test.sent_split	0.9974	0.9949	0.9962	High
it_partut-ud-train.sent_split	0.9663	0.9461	0.9561	Stable

Overall average Version 1: Precision 0.9724, Recall 0.9564, F1 0.9643.

5. Error analysis

After the first evaluation, errors were analyzed. Debug scripts were used to identify gold sentences not matched by Version 1. The analysis revealed a very clear pattern: many sentences in the dataset ended with the punctuation marks ':' and ';'.

Some non-matched examples:

- The narrator says:
- as Robb explained:
- family is the foundation of freedom, justice and peace in the world;
- Il narratore dice:
- come spiega Robb:
- fondamento della libertà, della giustizia e della pace nel mondo;

It was therefore necessary to extend the splitting rule to include the punctuation marks that often act as sentence boundaries in the dataset.

6. Version 2: extended punctuation

Version 2 is identical to Version 1 in everything, except for a single line in the segmentation regex (and, of course, the output directories).

Segmentation rule used in Version 2:

```
re.split(r'(?<=[.!?;:])\s+', testo)
```

Compared to Version 1, the system now also splits after ';' and ':'. All the rest of the code remained unchanged.

7. Evaluation of Version 2

Version 2 was also evaluated on the same files, using the same procedure. The comparison is therefore correct and direct.

Risult Version 2

File	Precision	Recall	F1	Note
en_partut-ud-dev.sent_split	0.9881	0.9940	0.9910	Very Improved
en_partut-ud-test.sent_split	1.0000	1.0000	1.0000	Perfect
en_partut-ud-train.sent_split	0.9881	0.9911	0.9896	Very high
it_partut-ud-dev.sent_split	0.9886	0.9943	0.9914	Very Improved
it_partut-ud-test.sent_split	1.0000	1.0000	1.0000	Perfect
it_partut-ud-train.sent_split	0.9904	0.9930	0.9917	Very High

Overall average Version 2: Precision 0.9925, Recall 0.9954, F1 0.9940.

8. Direct comparison between the two versions

The final comparison shows that Version 2 is clearly superior to Version 1. The difference does not come from a more complex system, but from a minimal modification guided by error analysis.

Version	Precision media	Recall media	F1 media
Versione 1	0.9724	0.9564	0.9643
Versione 2	0.9925	0.9954	0.9940

Version 2 consistently improves both English and Italian results.

9. Consideration about residual errors

Version 2 is still not perfect. Some difficult cases remain, especially with:

- abbreviations and initials, for example W. H. or T. S. Eliot;
- quotes and quotation marks;
- ellipses and sequences with ...;
- numbered structures such as 1. Definitions. or 8. Varie;
- editorial or non-standard sentence fragments.

However, the final results are satisfactory for a first approach.

10. Final conclusion

The project showed that, for this task and these datasets, a very simple baseline can already perform well. However, it also showed that a small, well-motivated modification based on error analysis can lead to a significant improvement.

Version 1: baseline system, simple and correct, based on . ! ?

Version 2: same system, but extended with ; :

Final improvement: average F1 from 0.9643 to 0.9940

This result is important not only for performance, but also from a methodological perspective: the improvement was achieved experimentally through error analysis and with a minimal modification to the original code.